

users.js — Fundamentos

Explicación detallada de la tabla de usuarios y sus conexiones (GymBro App)

Es la **primera y más importante** tabla de la base de datos simulada: contiene tanto a los **entrenadores** como a los **alumnos**, todos en el mismo array, diferenciados por un campo booleano. Es el archivo que consume directamente AuthContext.js para todo lo relacionado a login/registro.

1. La forma de los datos — un array con dos “tipos” de usuario mezclados

```
export const users = [
  {
    id: 1,
    isTrainer: true,
    name: 'Entrenador Mañana',
    initials: 'EM',
    firstName: 'Entrenador',
    lastName: 'Mañana',
    email: 'entrenadormaniana@gmail.com',
    dni: '30123456',
    password: 'gymbro123',
    gender: 'Masculino',
    age: '32 años',
    weight: '82 kg',
    height: '180 cm',
    phone: '3794112233',
    emergencyPhone: '3794112244',
    healthInsurance: 'OSDE',
    contraindications: 'Ninguna registrada',
    goal: 'Ganar fuerza',
  },
  {
    id: 2,
    isTrainer: false,
    id_rutina: 1,
    name: 'Sol Gaetmank',
    ...
  },
  ...
];
```

Hay 5 usuarios precargados: 2 entrenadores (id: 1 y id: 5) y 3 alumnos (id: 2, 3, 4). El campo clave que distingue el “tipo” de cada uno es:

- **isTrainer** (booleano): es el mismo campo que decide en app/index.tsx qué grupo de Stack.Screen mostrar (user.isTrainer ? <pantallas de entrenador> : <pantallas de alumno>). Todo el ruteo por rol de la app depende de este único campo.

Fijate que los entrenadores **no tienen** id_rutina (no aparece en su objeto), mientras que los alumnos sí. Esto es consistente: un entrenador no tiene una rutina asignada, es quien las crea para otros. JavaScript no exige que todos los objetos de un array tengan exactamente las mismas propiedades (a diferencia de una tabla SQL estricta con columnas fijas), así que esta “forma opcional” es válida, aunque hay que tener cuidado al leer user.id_rutina en un entrenador (daría undefined, no un error).

Todos los usuarios comparten la misma contraseña en los datos de ejemplo ('gymbro123') — es solo para facilitar las pruebas/demo, no una regla del sistema.

2. getUserById (líneas 107-109)

```
export function getUserById(userId) {
  return users.find((user) => user.id === userId);
}
```

El mismo patrón de siempre. Curiosamente, en el código revisado hasta ahora **no aparece usado directamente desde ninguna pantalla** — sí lo usa internamente setUserRoutine (se ve más abajo). Podría ser útil para futuras pantallas que necesiten buscar un usuario puntual por id.

3. addUser — el alta de un alumno nuevo (líneas 111-142)

```
export function addUser(data) {
  const firstName = data.firstName.trim();
  const lastName = data.lastName.trim();

  const newUser = {
    id: Math.max(...users.map((user) => user.id)) + 1,
    isTrainer: false,
    id_rutina: null,
    name: `${firstName} ${lastName}`,
    initials: `${firstName[0]}${lastName[0]}`.toUpperCase(),
    firstName,
    lastName,
    email: data.email.trim().toLowerCase(),
    dni: data.dni.trim(),
    password: data.password,
    gender: data.gender,
    age: `${data.age} años`,
    weight: `${data.weight} kg`,
    height: `${data.height} cm`,
    phone: data.phone.trim(),
    emergencyPhone: data.emergencyPhone.trim(),
    healthInsurance: data.healthInsurance.trim() || '-',
    contraindications: data.contraindications.trim() || 'Ninguna',
    goal: data.goal,
  };

  users.push(newUser);
  return newUser;
}
```

Este es el helper que llama register() de AuthContext.js cuando alguien completa el formulario de signup.js. Vale la pena desglosarlo:

- **id autoincremental:** `Math.max(...users.map(u => u.id)) + 1` — mismo patrón que createRoutine de routines.js. Toma todos los ids existentes, agarra el mayor, y le suma 1.
- **isTrainer: false fijo:** cualquiera que se registre desde signup.js es siempre un alumno. No hay forma de auto-registrarse como entrenador desde la UI — los dos entrenadores están hardcodedados en el array inicial. Es una decisión de diseño coherente con cómo funcionaría un gimnasio real: los entrenadores los da de alta el gimnasio, no se auto-registran.
- **id_rutina: null fijo:** todo alumno nace **sin rutina asignada**. El comentario del código lo dice explícito: “Nace sin rutina: se la arma el entrenador desde su pantalla de edición.” Esto conecta directo con routines.js: `getRoutineById(null)` no encuentra nada, y `mi_rutina_user.js` muestra “Todavía no tenés una rutina asignada” hasta que un entrenador la cree con `editar_rutina_trainer.js`.
- **name e initials derivados:** no los tipea el usuario, se **calculan** a partir de `firstName/lastName`. ``${firstName[0]}${lastName[0]}`` toma el primer carácter de cada uno (ej. “Sol” + “Gaetmank” →

“S” + “G”) y `.toUpperCase()` asegura que queden en mayúscula aunque el usuario haya tipeado el nombre en minúscula. Esas iniciales son las que se ven, por ejemplo, en avatares circulares en otras pantallas.

- **Normalización de strings:** `email.trim().toLowerCase()` (mismo criterio que usa `login()` al comparar), `dni.trim()`, `phone.trim()`, etc. — limpia espacios accidentales del formulario.
- **Formato con unidad agregado:** `age: `${data.age} años`, weight: `${data.weight} kg`, height: `${data.height} cm``. En `signup.js`, `handleRegister` ya había convertido `age/weight/height` a números validados y los había vuelto a pasar como string plano (`String(age)`) antes de llamar a `register(...)`. Acá, `addUser` les agrega la unidad correspondiente para que el dato quede guardado con el mismo formato que los usuarios de ejemplo ('25 años', '70.5 kg', '175 cm'), listo para mostrarse directamente en la UI sin tener que recordar la unidad en cada pantalla que lo muestre.
- **Opcionales con valor por defecto:** `healthInsurance.trim() || '-'` y `contraindications.trim() || 'Ninguna'` — si el campo llega vacío (en `signup.js` esos dos campos eran opcionales, no estaban en `REQUIRED_FIELDS`), se guarda un placeholder en vez de un string vacío, para que se vea prolijo en cualquier pantalla de perfil.

4. setUserRoutine — el vínculo alumno ↔ rutina (líneas 144-150)

```
export function setUserRoutine(userId, routineId) {
  const user = getUserById(userId);
  if (!user) return false;
  user.id_rutina = routineId;
  return true;
}
```

Es la función que llama `editar_rutina_trainer.js` justo después de `createRoutine(...)`, para completar el vínculo. Fijate que **muta el objeto user directamente** (`user.id_rutina = routineId`) en vez de reemplazar el array `users` completo — mismo criterio de “mutación in-place sobre datos de módulo” que `updateRoutineDays`.

5. Dónde se usa users como array completo (no por id)

`buscar_usuario_trainer.js` — la pantalla donde el entrenador busca alumnos:

```
const usuariosFiltrados = users
  .filter((user) => !user.isTrainer)
  .filter((user) =>
    normalizar(`${user.firstName} ${user.lastName} ${user.email}`).includes(
      normalizar(filtro)
    )
  );
```

Dos `.filter()` encadenados:

- **!user.isTrainer:** excluye a los entrenadores del listado — esta pantalla es solo para que un entrenador busque **alumnos**, no a otros entrenadores.
- **Coincidencia de texto:** igual que el buscador de ejercicios de `editar_rutina_trainer.js`, concatena `firstName + lastName + email`, normaliza (sin tildes, minúsculas) y verifica si contiene el texto escrito en el buscador (`filtro`).

Es una búsqueda **100% del lado del cliente**: no hay ninguna llamada a un servidor, simplemente filtra el array en memoria en cada render.

6. El grafo completo de relaciones (actualizado con las 3 tablas)

```

        login() compara email+password
            |
users.js  --id_rutina--> routines.js  --items.exerciseId--> exercises.js
    ^                               |
    +----- setUserRoutine -----+

```

Resumen para el oral

“users.js es la tabla central de usuarios: entrenadores y alumnos conviven en el mismo array, distinguidos por isTrainer, campo que gobierna todo el ruteo por rol de la app. getUserById busca por id, addUser da de alta un alumno nuevo desde signup.js — siempre con isTrainer: false e id_rutina: null, calculando name/initials a partir del nombre y agregando las unidades de medida a los datos físicos — y setUserRoutine completa el vínculo entrenador→alumno→rutina la primera vez que se guarda una rutina. buscar_usuario_trainer.js filtra este array del lado del cliente para excluir entrenadores y buscar por texto, sin ningún llamado a un backend.”